

Hazard Unit UVM Verification

Generated by Doxygen 1.16.1

1 Topic Index	1
1.1 Topics	1
2 Directory Hierarchy	3
2.1 Directories	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Topic Documentation	11
6.1 RTL design	11
6.1.1 Detailed Description	11
6.2 Modules	11
6.3 Testbench components	11
6.3.1 Detailed Description	11
6.4 Interfaces	12
6.4.1 Detailed Description	12
6.4.2 Function Documentation	12
6.4.2.1 s_interface()	12
6.5 UVM verification components	13
6.5.1 Detailed Description	13
7 Directory Documentation	15
7.1 filter Directory Reference	15
7.1.1 Detailed Description	15
7.1.2 SystemVerilog Doxygen filter	15
7.2 reference_model Directory Reference	15
7.3 rtl Directory Reference	15
7.4 tb Directory Reference	16
7.5 uvm Directory Reference	16
8 Class Documentation	17
8.1 agent Class Reference	17
8.1.1 Detailed Description	18
8.1.2 Member Function Documentation	18
8.1.2.1 build_phase()	18
8.1.2.2 connect_phase()	18
8.1.2.3 new()	18
8.1.2.4 uvm_component_utils()	19

8.1.3 Member Data Documentation	19
8.1.3.1 m_monitor	19
8.1.3.2 m_sequencer	19
8.2 coverage Class Reference	19
8.2.1 Detailed Description	20
8.2.2 Member Function Documentation	20
8.2.2.1 cg_hazard_unit()	20
8.2.2.2 new()	21
8.2.2.3 uvm_component_utils()	21
8.2.2.4 write()	21
8.3 driver Class Reference	21
8.3.1 Detailed Description	22
8.3.2 Member Function Documentation	23
8.3.2.1 build_phase()	23
8.3.2.2 drive_item()	23
8.3.2.3 new()	23
8.3.2.4 run_phase()	23
8.3.2.5 uvm_component_utils()	23
8.3.3 Member Data Documentation	23
8.3.3.1 req_pkt	23
8.4 env Class Reference	24
8.4.1 Detailed Description	25
8.4.2 Member Function Documentation	25
8.4.2.1 build_phase()	25
8.4.2.2 connect_phase()	25
8.4.2.3 new()	25
8.4.2.4 uvm_component_utils()	25
8.4.3 Member Data Documentation	25
8.4.3.1 m_coverage	25
8.4.3.2 m_scoreboard	26
8.5 hazard_unit_sequence Class Reference	26
8.5.1 Detailed Description	27
8.5.2 Member Function Documentation	27
8.5.2.1 body()	27
8.5.2.2 new()	27
8.5.2.3 uvm_object_utils()	27
8.5.3 Member Data Documentation	27
8.5.3.1 num_packets	27
8.6 monitor Class Reference	28
8.6.1 Detailed Description	29
8.6.2 Member Function Documentation	29
8.6.2.1 build_phase()	29

8.6.2.2 new()	29
8.6.2.3 run_phase()	29
8.6.2.4 uvm_component_utils()	29
8.6.3 Member Data Documentation	29
8.6.3.1 mon_analysis_port	29
8.7 pkt Class Reference	30
8.7.1 Detailed Description	31
8.7.2 Member Function Documentation	31
8.7.2.1 new()	31
8.7.2.2 uvm_field_int() [1/8]	32
8.7.2.3 uvm_field_int() [2/8]	32
8.7.2.4 uvm_field_int() [3/8]	32
8.7.2.5 uvm_field_int() [4/8]	32
8.7.2.6 uvm_field_int() [5/8]	32
8.7.2.7 uvm_field_int() [6/8]	32
8.7.2.8 uvm_field_int() [7/8]	32
8.7.2.9 uvm_field_int() [8/8]	33
8.7.2.10 uvm_object_utils_begin()	33
8.7.3 Member Data Documentation	33
8.7.3.1 i_pcSrc_EX	33
8.7.3.2 i_rdAddr_EX	33
8.7.3.3 i_rdAddr_M	33
8.7.3.4 i_rdAddr_WB	33
8.7.3.5 i_reg_write_M	33
8.7.3.6 i_reg_write_WB	34
8.7.3.7 i_result_src_EX	34
8.7.3.8 i_rs1Addr_EX	34
8.7.3.9 i_rs1Addr_ID	34
8.7.3.10 i_rs2Addr_EX	34
8.7.3.11 i_rs2Addr_ID	34
8.7.3.12 o_flush_EX	34
8.7.3.13 o_flush_ID	34
8.7.3.14 o_forward_rs1_EX	35
8.7.3.15 o_forward_rs2_EX	35
8.7.3.16 o_stall_ID	35
8.7.3.17 o_stall_IF	35
8.8 s_interface Interface Reference	35
8.8.1 Detailed Description	35
8.9 scoreboard Class Reference	36
8.9.1 Detailed Description	37
8.9.2 Member Function Documentation	37
8.9.2.1 build_phase()	37

8.9.2.2 check_phase()	37
8.9.2.3 new()	37
8.9.2.4 run_phase()	37
8.9.2.5 uvm_component_utils()	37
8.9.2.6 write()	38
8.9.3 Member Data Documentation	38
8.9.3.1 ap_imp	38
8.9.3.2 num_errors	38
8.10 sequencer Class Reference	38
8.10.1 Detailed Description	39
8.10.2 Member Function Documentation	39
8.10.2.1 uvm_component_utils()	39
8.10.3 Member Data Documentation	39
8.10.3.1 parent	39
8.11 test Class Reference	40
8.11.1 Detailed Description	41
8.11.2 Member Function Documentation	41
8.11.2.1 build_phase()	41
8.11.2.2 end_of_elaboration_phase()	41
8.11.2.3 new()	41
8.11.2.4 run_phase()	41
8.11.2.5 uvm_component_utils()	41
8.11.3 Member Data Documentation	41
8.11.3.1 m_sequence	41
9 File Documentation	43
9.1 filter/README.md File Reference	43
9.2 reference_model/reference_model.c File Reference	43
9.2.1 Detailed Description	44
9.2.2 Function Documentation	44
9.2.2.1 hazard_unit_golden()	44
9.3 rtl/hazard_unit.v File Reference	45
9.3.1 Function Documentation	45
9.3.1.1 if() [1/4]	45
9.3.1.2 if() [2/4]	46
9.3.1.3 if() [3/4]	46
9.3.1.4 if() [4/4]	46
9.3.2 Variable Documentation	46
9.3.2.1 i_pcSrc_EX	46
9.3.2.2 i_rdAddr_EX	46
9.3.2.3 i_rdAddr_M	46
9.3.2.4 i_rdAddr_WB	46

9.3.2.5 i_reg_write_M	46
9.3.2.6 i_reg_write_WB	46
9.3.2.7 i_result_src_EX	47
9.3.2.8 i_rs1Addr_EX	47
9.3.2.9 i_rs1Addr_ID	47
9.3.2.10 i_rs2Addr_EX	47
9.3.2.11 i_rs2Addr_ID	47
9.3.2.12 load_hazard_detect	47
9.3.2.13 o_flush_EX	47
9.3.2.14 o_flush_ID	47
9.3.2.15 o_forward_rs1_EX	47
9.3.2.16 o_forward_rs2_EX	47
9.3.2.17 o_stall_ID	48
9.3.2.18 o_stall_IF	48
9.4 rtl/hazard_unit_wrapper.sv File Reference	48
9.4.1 Detailed Description	48
9.5 tb/s_interface.sv File Reference	48
9.5.1 Detailed Description	48
9.6 tb/tb_top.sv File Reference	48
9.6.1 Detailed Description	49
9.6.2 Function Documentation	49
9.6.2.1 run_test()	49
9.6.2.2 U_HAZARD_UNIT_WRAPPER()	49
9.6.2.3 vif()	49
9.6.3 Variable Documentation	49
9.6.3.1 forever	49
9.6.3.2 rst	50
9.7 uvm/agent.sv File Reference	50
9.7.1 Detailed Description	50
9.8 agent.sv	50
9.9 uvm/coverage.sv File Reference	51
9.9.1 Detailed Description	51
9.10 coverage.sv	52
9.11 uvm/driver.sv File Reference	52
9.11.1 Detailed Description	53
9.12 driver.sv	53
9.13 uvm/env.sv File Reference	54
9.13.1 Detailed Description	54
9.14 env.sv	55
9.15 uvm/hazard_unit_pkg.sv File Reference	55
9.15.1 Detailed Description	55
9.16 uvm/hazard_unit_sequence.sv File Reference	56

9.16.1 Detailed Description	56
9.17 hazard_unit_sequence.sv	56
9.18 uvm/monitor.sv File Reference	57
9.18.1 Detailed Description	57
9.19 monitor.sv	57
9.20 uvm/pkt.sv File Reference	58
9.20.1 Detailed Description	59
9.21 pkt.sv	59
9.22 uvm/scoreboard.sv File Reference	59
9.22.1 Detailed Description	60
9.22.2 Function Documentation	60
9.22.2.1 hazard_unit_golden()	60
9.23 scoreboard.sv	61
9.24 uvm/sequencer.sv File Reference	63
9.24.1 Detailed Description	63
9.25 sequencer.sv	63
9.26 uvm/test.sv File Reference	64
9.26.1 Detailed Description	64
9.27 test.sv	64
Index	65

Chapter 1

Topic Index

1.1 Topics

Here is a list of all topics with brief descriptions:

RTL design	11
Modules	11
Testbench components	11
Interfaces	12
UVM verification components	13

Chapter 2

Directory Hierarchy

2.1 Directories

filter	15
reference_model	15
reference_model.c	43
rtl	15
hazard_unit.v	45
hazard_unit_wrapper.sv	48
tb	16
s_interface.sv	48
tb_top.sv	48
uvm	16
agent.sv	50
coverage.sv	51
driver.sv	52
env.sv	54
hazard_unit_pkg.sv	55
hazard_unit_sequence.sv	56
monitor.sv	57
pkt.sv	58
scoreboard.sv	59
sequencer.sv	63
test.sv	64

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

s_interface	35
uvm_agent	
agent	17
uvm_driver	
driver	21
uvm_env	
env	24
uvm_monitor	
monitor	28
uvm_scoreboard	
scoreboard	36
uvm_sequence	
hazard_unit_sequence	26
uvm_sequence_item	
pkt	30
uvm_sequencer	
sequencer	38
uvm_subscriber	
coverage	19
uvm_test	
test	40

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

agent	UVM agent that connects the sequencer, driver, and monitor	17
coverage	UVM coverage subscriber for hazard-unit transaction sampling	19
driver	UVM driver for hazard-unit transactions	21
env	Top-level UVM environment for the hazard-unit verification bench	24
hazard_unit_sequence	UVM sequence that drives hazard-unit transactions until coverage reaches the target	26
monitor	UVM monitor for hazard-unit transaction observation	28
pkt	Sequence item that models one hazard-unit evaluation (stimulus + expected response fields)	30
s_interface	Hazard-unit stimulus/response interface shared by the DUT and the UVM environment	35
scoreboard	UVM scoreboard for hazard-unit functional checking	36
sequencer	UVM sequencer for hazard-unit transactions	38
test	Top-level UVM test for the hazard-unit verification bench	40

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

reference_model/ reference_model.c	
Golden reference model for the hazard unit, called from the UVM scoreboard via DPI-C	43
rtl/ hazard_unit.v	45
rtl/ hazard_unit_wrapper.sv	
Thin combinational wrapper around hazard_unit , used as the DUT	48
tb/ s_interface.sv	
Hazard-unit stimulus/response interface shared by the DUT and the UVM environment	48
tb/ tb_top.sv	
Top-level testbench that instantiates the DUT, interface, and UVM test	48
uvm/ agent.sv	
UVM agent that bundles the sequencer, driver, and monitor	50
uvm/ coverage.sv	
Coverage collector for hazard-unit transactions	51
uvm/ driver.sv	
UVM driver that translates hazard-unit transactions into interface signal activity	52
uvm/ env.sv	
UVM environment that instantiates the agent, scoreboard, and coverage collector	54
uvm/ hazard_unit_pkg.sv	
Package that includes the UVM classes used by the hazard-unit verification environment	55
uvm/ hazard_unit_sequence.sv	
UVM sequence that drives hazard-unit transactions until coverage reaches the target	56
uvm/ monitor.sv	
UVM monitor that samples hazard-unit transactions and publishes them to subscribers	57
uvm/ pkt.sv	
Transaction item used to model one hazard-unit evaluation in the UVM environment	58
uvm/ scoreboard.sv	
Scoreboard that checks hazard-unit transactions against the golden reference model	59
uvm/ sequencer.sv	
UVM sequencer that provides transactions to the driver	63
uvm/ test.sv	
Top-level UVM test that builds the environment and starts the main sequence	64

Chapter 6

Topic Documentation

6.1 RTL design

Register-transfer-level implementation of the hazard-unit DUT.

Files

- file [hazard_unit_wrapper.sv](#)
Thin combinational wrapper around [hazard_unit](#), used as the DUT.

6.1.1 Detailed Description

Register-transfer-level implementation of the hazard-unit DUT.

6.2 Modules

6.3 Testbench components

Interface and top-level testbench modules used by the verification environment.

Files

- file [s_interface.sv](#)
Hazard-unit stimulus/response interface shared by the DUT and the UVM environment.
- file [tb_top.sv](#)
Top-level testbench that instantiates the DUT, interface, and UVM test.

6.3.1 Detailed Description

Interface and top-level testbench modules used by the verification environment.

6.4 Interfaces

Functions

- interface [s_interface](#) (input logic clk, input logic reset)

6.4.1 Detailed Description

6.4.2 Function Documentation

6.4.2.1 s_interface()

```
interface s_interface (
    input logic clk,
    input logic reset)
```

< rs1 address at Decode ([i_rs1Addr_ID](#)).

< rs2 address at Decode ([i_rs2Addr_ID](#)).

< rd address at Execute ([i_rdAddr_EX](#)).

< rs1 address at Execute ([i_rs1Addr_EX](#)).

< rs2 address at Execute ([i_rs2Addr_EX](#)).

< PC-redirect flag at Execute ([i_pcSrc_EX](#)).

< Write-back source select at Execute ([i_result_src_EX](#)).

< rd address at Memory ([i_rdAddr_M](#)).

< Register-file write enable at Memory ([i_reg_write_M](#)).

< rd address at WriteBack ([i_rdAddr_WB](#)).

< Register-file write enable at WriteBack ([i_reg_write_WB](#)).

< [o_stall_IF](#), driven by the DUT.

< [o_stall_ID](#), driven by the DUT.

< [o_flush_ID](#), driven by the DUT.

< [o_flush_EX](#), driven by the DUT.

< [o_forward_rs1_EX](#), driven by the DUT.

< [o_forward_rs2_EX](#), driven by the DUT.

< Pulsed high by the driver while a transaction is valid.

Modport used by the UVM driver: drives the stimulus signals.

Modport used by the UVM monitor: observes every signal, drives none.

Modport used by the [hazard_unit_wrapper](#) DUT: consumes the stimulus, drives the results.

6.5 UVM verification components

UVM classes and utilities used to verify the hazard unit.

Files

- file [reference_model.c](#)
Golden reference model for the hazard unit, called from the UVM scoreboard via DPI-C.
- file [agent.sv](#)
UVM agent that bundles the sequencer, driver, and monitor.
- file [coverage.sv](#)
Coverage collector for hazard-unit transactions.
- file [driver.sv](#)
UVM driver that translates hazard-unit transactions into interface signal activity.
- file [env.sv](#)
UVM environment that instantiates the agent, scoreboard, and coverage collector.
- file [hazard_unit_sequence.sv](#)
UVM sequence that drives hazard-unit transactions until coverage reaches the target.
- file [monitor.sv](#)
UVM monitor that samples hazard-unit transactions and publishes them to subscribers.
- file [pkt.sv](#)
Transaction item used to model one hazard-unit evaluation in the UVM environment.
- file [scoreboard.sv](#)
Scoreboard that checks hazard-unit transactions against the golden reference model.
- file [sequencer.sv](#)
UVM sequencer that provides transactions to the driver.
- file [test.sv](#)
Top-level UVM test that builds the environment and starts the main sequence.

6.5.1 Detailed Description

UVM classes and utilities used to verify the hazard unit.

Chapter 7

Directory Documentation

7.1 filter Directory Reference

7.1.1 Detailed Description

7.1.2 SystemVerilog Doxygen filter

`idv_doxyfilter_sv.pl` converts SystemVerilog into C++-like code so Doxygen can parse it.

- **Author:** Sean O'Boyle, Intelligent Design Verification
- **Source:** <https://github.com/SeanOBoyle/DoxygenFilterSystemVerilog>
- **License:** GNU GPL v3 or later (see the header of the script). The file is included here unmodified.

It is wired up through `FILTER_PATTERNS` in `../Doxyfile`; run `doxygen` from the parent directory.

7.2 reference_model Directory Reference

Files

- file [reference_model.c](#)
Golden reference model for the hazard unit, called from the UVM scoreboard via DPI-C.

7.3 rtl Directory Reference

Files

- file [hazard_unit.v](#)
- file [hazard_unit_wrapper.sv](#)
Thin combinational wrapper around [hazard_unit](#), used as the DUT.

7.4 tb Directory Reference

Files

- file [s_interface.sv](#)
Hazard-unit stimulus/response interface shared by the DUT and the UVM environment.
- file [tb_top.sv](#)
Top-level testbench that instantiates the DUT, interface, and UVM test.

7.5 uvm Directory Reference

Files

- file [agent.sv](#)
UVM agent that bundles the sequencer, driver, and monitor.
- file [coverage.sv](#)
Coverage collector for hazard-unit transactions.
- file [driver.sv](#)
UVM driver that translates hazard-unit transactions into interface signal activity.
- file [env.sv](#)
UVM environment that instantiates the agent, scoreboard, and coverage collector.
- file [hazard_unit_pkg.sv](#)
Package that includes the UVM classes used by the hazard-unit verification environment.
- file [hazard_unit_sequence.sv](#)
UVM sequence that drives hazard-unit transactions until coverage reaches the target.
- file [monitor.sv](#)
UVM monitor that samples hazard-unit transactions and publishes them to subscribers.
- file [pkt.sv](#)
Transaction item used to model one hazard-unit evaluation in the UVM environment.
- file [scoreboard.sv](#)
Scoreboard that checks hazard-unit transactions against the golden reference model.
- file [sequencer.sv](#)
UVM sequencer that provides transactions to the driver.
- file [test.sv](#)
Top-level UVM test that builds the environment and starts the main sequence.

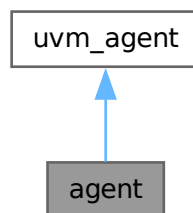
Chapter 8

Class Documentation

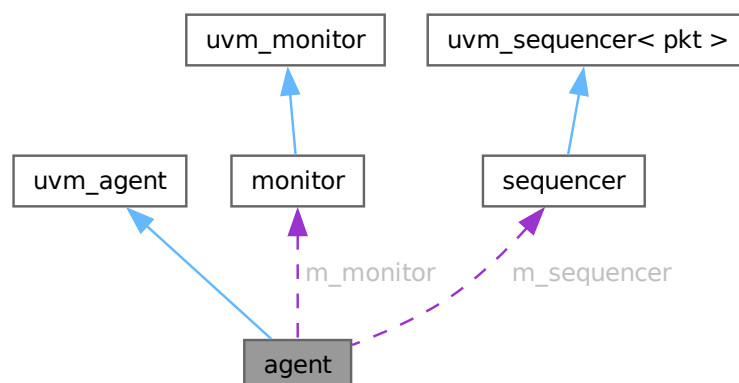
8.1 agent Class Reference

UVM agent that connects the sequencer, driver, and monitor.

Inheritance diagram for agent:



Collaboration diagram for agent:



Public Member Functions

- [uvm_component_utils](#) ([agent](#)) [driver](#) m_driver
Drives sequence items onto the DUT interface (null when passive).
- [new](#) (string name="agent", uvm_component parent=null)
- virtual void [build_phase](#) (uvm_phase phase)
Builds the sequencer/driver (active agent only) and the monitor.
- virtual void [connect_phase](#) (uvm_phase phase)
Connects the driver's seq_item port to the sequencer (active agent only).

Public Attributes

- [monitor](#) m_monitor
Samples DUT activity and publishes it (always created).
- [sequencer](#) m_sequencer
Arbitrates sequence items for the driver (null when passive).

8.1.1 Detailed Description

UVM agent that connects the sequencer, driver, and monitor.

Coordinates the active and passive verification components for the hazard-unit testbench and exposes the common sequence and analysis interfaces. The sequencer and driver are only created when the agent is active; the monitor is always created.

8.1.2 Member Function Documentation

8.1.2.1 build_phase()

```
virtual void agent::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Builds the sequencer/driver (active agent only) and the monitor.

8.1.2.2 connect_phase()

```
virtual void agent::connect_phase (  
    uvm_phase phase) [inline], [virtual]
```

Connects the driver's seq_item port to the sequencer (active agent only).

8.1.2.3 new()

```
agent::new (  
    string name = "agent",  
    uvm_component parent = null) [inline]
```

8.1.2.4 uvm_component_utils()

```
agent::uvm_component_utils (
    agent )
```

Drives sequence items onto the DUT interface (null when passive).

8.1.3 Member Data Documentation

8.1.3.1 m_monitor

```
monitor agent::m_monitor
```

Samples DUT activity and publishes it (always created).

8.1.3.2 m_sequencer

```
sequencer agent::m_sequencer
```

Arbitrates sequence items for the driver (null when passive).

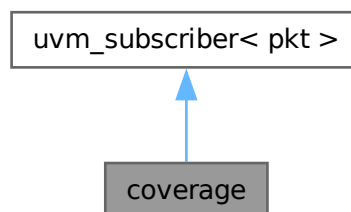
The documentation for this class was generated from the following file:

- [uvm/agent.sv](#)

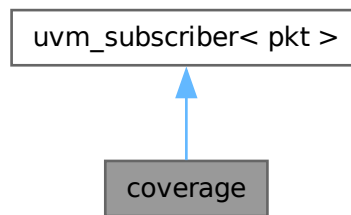
8.2 coverage Class Reference

UVM coverage subscriber for hazard-unit transaction sampling.

Inheritance diagram for coverage:



Collaboration diagram for coverage:



Public Member Functions

- `uvm_component_utils (coverage) pkt tr`
Transaction currently being sampled by `cg_hazard_unit`.
- `covergroup cg_hazard_unit ()`
Forwarding/hazard condition coverage model.
- `new` (string name="coverage", uvm_component parent)
- virtual void `write (pkt t)`
Samples the covergroup for one transaction and republishes the running coverage percentage.

8.2.1 Detailed Description

UVM coverage subscriber for hazard-unit transaction sampling.

Tracks each hazard/forwarding condition individually, plus their priority-order combinations, and publishes the current percentage to the sequence layer through the configuration database under the key "cov_status".

8.2.2 Member Function Documentation

8.2.2.1 `cg_hazard_unit()`

```
covergroup coverage::cg_hazard_unit () [inline]
```

Forwarding/hazard condition coverage model.

All 4 write-back source codes (2'b10 is the load/"data from memory" code the load-use check should key on).

Both PC-redirect states, to exercise `o_flush_ID/o_flush_EX`.

True when the Decode-stage rs1 matches the Execute-stage rd while the Execute-stage instruction is a load: the load-use hazard condition for rs1.

Same as `cp_load_hazard_rs1`, for rs2.

True when rs1 at Execute matches rd at Memory, that Memory instruction writes the register file, and rs1 isn't x0: the Memory-forward condition for rs1.

Same as `cp_forward_rs1_mem`, against the WriteBack stage.

Same as `cp_forward_rs1_mem`, for rs2.

Same as `cp_forward_rs1_wb`, for rs2.

Crossing mem/wb for rs1 exercises the mem-over-wb priority when both are true.

Crossing mem/wb for rs2 exercises the mem-over-wb priority when both are true.

8.2.2.2 new()

```
coverage::new (
    string name = "coverage",
    uvm_component parent) [inline]
```

8.2.2.3 uvm_component_utils()

```
coverage::uvm_component_utils (
    coverage )
```

Transaction currently being sampled by [cg_hazard_unit](#).

8.2.2.4 write()

```
virtual void coverage::write (
    pkt t) [inline], [virtual]
```

Samples the covergroup for one transaction and republishes the running coverage percentage.

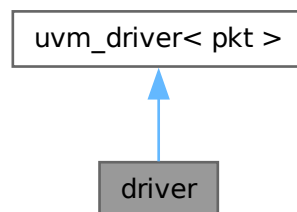
The documentation for this class was generated from the following file:

- [uvm/coverage.sv](#)

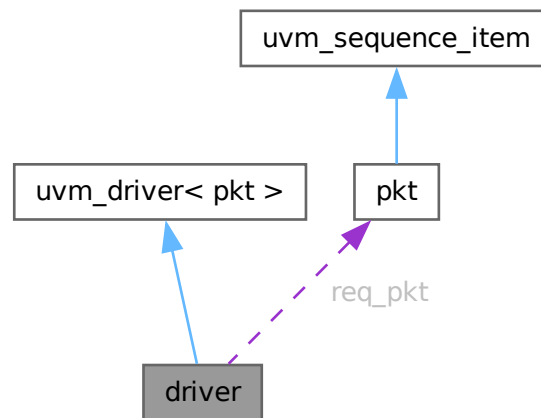
8.3 driver Class Reference

UVM driver for hazard-unit transactions.

Inheritance diagram for driver:



Collaboration diagram for driver:



Public Member Functions

- [uvm_component_utils](#) ([driver](#)) virtual s_interface.driver [vif](#)
Driver modport of the DUT interface, obtained from the config DB (key "[vif](#)").
- [new](#) (string name="driver", uvm_component parent)
- virtual void [build_phase](#) (uvm_phase phase)
Fetches the virtual interface handle set by the top-level testbench.
- virtual task [run_phase](#) (uvm_phase phase)
Waits for reset to be released, then drives sequence items forever.
- virtual task [drive_item](#) ([pkt](#) [pkt_item](#))
Drives one full set of pipeline-register addresses/control bits for one clock cycle.

Public Attributes

- [pkt](#) [req_pkt](#)
Sequence item currently being driven.

8.3.1 Detailed Description

UVM driver for hazard-unit transactions.

Converts each transaction item into the corresponding stimulus pulse on [s_interface](#). Requires the virtual interface to be set in the configuration database under the key "[vif](#)".

8.3.2 Member Function Documentation

8.3.2.1 build_phase()

```
virtual void driver::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fetches the virtual interface handle set by the top-level testbench.

8.3.2.2 drive_item()

```
virtual task driver::drive_item (  
    pkt pkt_item) [inline], [virtual]
```

Drives one full set of pipeline-register addresses/control bits for one clock cycle.

8.3.2.3 new()

```
driver::new (  
    string name = "driver",  
    uvm_component parent) [inline]
```

8.3.2.4 run_phase()

```
virtual task driver::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Waits for reset to be released, then drives sequence items forever.

8.3.2.5 uvm_component_utils()

```
driver::uvm_component_utils (  
    driver )
```

Driver modport of the DUT interface, obtained from the config DB (key "[vif](#)").

8.3.3 Member Data Documentation

8.3.3.1 req_pkt

```
pkt driver::req_pkt
```

Sequence item currently being driven.

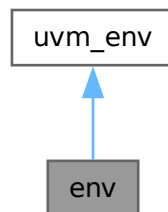
The documentation for this class was generated from the following file:

- [uvm/driver.sv](#)

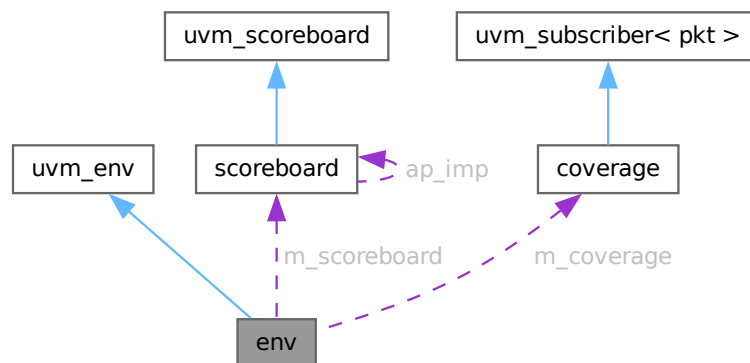
8.4 env Class Reference

Top-level UVM environment for the hazard-unit verification bench.

Inheritance diagram for env:



Collaboration diagram for env:



Public Member Functions

- `uvm_component_utils (env) agent m_agent`
Agent that drives and monitors the DUT.
- `new (string name="env", uvm_component parent=null)`
- `virtual void build_phase (uvm_phase phase)`
Creates the agent, scoreboard, and coverage collector.
- `virtual void connect_phase (uvm_phase phase)`
Connects the monitor's analysis port to both the scoreboard and coverage collector.

Public Attributes

- [scoreboard m_scoreboard](#)
Checks monitored transactions against the reference model.
- [coverage m_coverage](#)
Collects functional coverage of monitored transactions.

8.4.1 Detailed Description

Top-level UVM environment for the hazard-unit verification bench.

Instantiates the agent, scoreboard, and coverage collector and connects the monitored transactions to the checkers.

8.4.2 Member Function Documentation

8.4.2.1 build_phase()

```
virtual void env::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Creates the agent, scoreboard, and coverage collector.

8.4.2.2 connect_phase()

```
virtual void env::connect_phase (  
    uvm_phase phase) [inline], [virtual]
```

Connects the monitor's analysis port to both the scoreboard and coverage collector.

8.4.2.3 new()

```
env::new (  
    string name = "env",  
    uvm_component parent = null) [inline]
```

8.4.2.4 uvm_component_utils()

```
env::uvm_component_utils (  
    env )
```

Agent that drives and monitors the DUT.

8.4.3 Member Data Documentation

8.4.3.1 m_coverage

```
coverage env::m_coverage
```

Collects functional coverage of monitored transactions.

8.4.3.2 m_scoreboard

`scoreboard env::m_scoreboard`

Checks monitored transactions against the reference model.

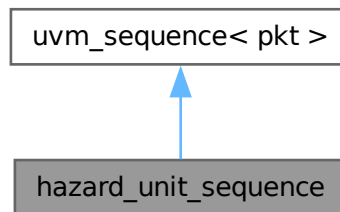
The documentation for this class was generated from the following file:

- `uvm/env.sv`

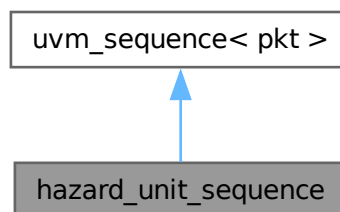
8.5 hazard_unit_sequence Class Reference

UVM sequence that drives hazard-unit transactions until coverage reaches the target.

Inheritance diagram for `hazard_unit_sequence`:



Collaboration diagram for `hazard_unit_sequence`:



Public Member Functions

- `uvm_object_utils (hazard_unit_sequence)` `real current_coverage=0`
Latest coverage percentage reported by the coverage collector.
- `new` (string name="hazard_unit_sequence")
- virtual task `body` ()
Sends randomized hazard-unit transactions until the coverage collector reports 100%.

Public Attributes

- int `num_packets` = 0
Number of transactions sent so far in this sequence.

8.5.1 Detailed Description

UVM sequence that drives hazard-unit transactions until coverage reaches the target.

Repeatedly randomizes and sends a transaction, polling the coverage collector after each one, until 100% coverage is reached. Reads the coverage percentage from the configuration database key "`cov_status`".

8.5.2 Member Function Documentation

8.5.2.1 `body()`

```
virtual task hazard_unit_sequence::body () [inline], [virtual]
```

Sends randomized hazard-unit transactions until the coverage collector reports 100%.

8.5.2.2 `new()`

```
hazard_unit_sequence::new (  
    string name = "hazard_unit_sequence") [inline]
```

8.5.2.3 `uvm_object_utils()`

```
hazard_unit_sequence::uvm_object_utils (  
    hazard_unit_sequence ) [pure virtual]
```

Latest coverage percentage reported by the coverage collector.

8.5.3 Member Data Documentation

8.5.3.1 `num_packets`

```
int hazard_unit_sequence::num_packets = 0
```

Number of transactions sent so far in this sequence.

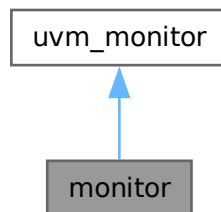
The documentation for this class was generated from the following file:

- [uvm/hazard_unit_sequence.sv](#)

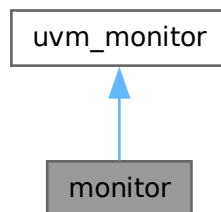
8.6 monitor Class Reference

UVM monitor for hazard-unit transaction observation.

Inheritance diagram for monitor:



Collaboration diagram for monitor:



Public Member Functions

- `uvm_component_utils (monitor)` virtual s_interface.monitor `vif`
Monitor modport of the DUT interface, obtained from the config DB (key "`vif`").
- `new` (string name="monitor", uvm_component parent=null)
- virtual void `build_phase` (uvm_phase phase)
Fetches the virtual interface handle set by the top-level testbench.
- virtual task `run_phase` (uvm_phase phase)
Samples one hazard-unit transaction per iteration and publishes it.

Public Attributes

- `uvm_analysis_port< pkt > mon_analysis_port`
Publishes each observed transaction to the scoreboard and coverage collector.

8.6.1 Detailed Description

UVM monitor for hazard-unit transaction observation.

Samples the DUT-visible signals on each transfer and publishes the resulting transaction object to downstream analysis components. Requires the virtual interface to be set in the configuration database under the key "[vif](#)".

8.6.2 Member Function Documentation

8.6.2.1 build_phase()

```
virtual void monitor::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fetches the virtual interface handle set by the top-level testbench.

8.6.2.2 new()

```
monitor::new (  
    string name = "monitor",  
    uvm_component parent = null) [inline]
```

8.6.2.3 run_phase()

```
virtual task monitor::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Samples one hazard-unit transaction per iteration and publishes it.

[hazard_unit_wrapper](#) is purely combinational, but its outputs only settle once the DUT's always @(*)/↵ continuous-assign logic reacts to the new inputs, which is not guaranteed to have happened yet in the same simulation step that wakes this process on `enable`'s rising edge. A minimal #1 delay lets that propagation finish before the outputs are sampled.

8.6.2.4 uvm_component_utils()

```
monitor::uvm_component_utils (  
    monitor )
```

Monitor modport of the DUT interface, obtained from the config DB (key "[vif](#)").

8.6.3 Member Data Documentation

8.6.3.1 mon_analysis_port

```
uvm_analysis_port<pkt> monitor::mon_analysis_port
```

Publishes each observed transaction to the scoreboard and coverage collector.

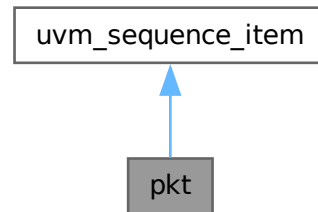
The documentation for this class was generated from the following file:

- [uvm/monitor.sv](#)

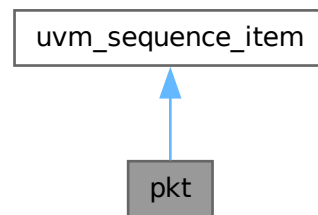
8.7 pkt Class Reference

Sequence item that models one hazard-unit evaluation (stimulus + expected response fields).

Inheritance diagram for pkt:



Collaboration diagram for pkt:



Public Member Functions

- [uvm_object_utils_begin](#) (pkt) [uvm_field_int](#)(i_rs1Addr_ID
- UVM_DEFAULT [uvm_field_int](#) (i_rs2Addr_ID, UVM_DEFAULT) [uvm_field_int](#)(i_rdAddr_EX
- UVM_DEFAULT UVM_DEFAULT [uvm_field_int](#) (i_rs1Addr_EX, UVM_DEFAULT) [uvm_field_int](#)(i_rs2Addr_EX
- UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT [uvm_field_int](#) (i_pcSrc_EX, UVM_DEFAULT) [uvm_field_↔](#)
[int](#)(i_result_src_EX
- UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT [uvm_field_int](#) (i_rdAddr_M, UVM_↔
DEFAULT) [uvm_field_int](#)(i_reg_write_M
- UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT [uvm_field_int](#) (i_rdAddr_WB,
- UVM_DEFAULT) [uvm_field_int](#)(i_reg_write_WB
- UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT
- [uvm_field_int](#) (o_stall_IF, UVM_DEFAULT) [uvm_field_int](#)(o_stall_ID
- UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_↔
_DEFAULT [uvm_field_int](#) (o_flush_ID, UVM_DEFAULT) [uvm_field_int](#)(o_flush_EX
- UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT
- UVM_DEFAULT UVM_DEFAULT [uvm_field_int](#) (o_forward_rs1_EX, UVM_DEFAULT) [uvm_field_↔](#)
[int](#)(o_forward_rs2_EX
- UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_↔
_DEFAULT UVM_DEFAULT UVM_DEFAULT [uvm_object_utils_end](#) [new](#) (string name="pkt")

Public Attributes

- rand logic< 3:0 > [i_rs1Addr_ID](#)
rs1 address at Decode.
- rand logic< 3:0 > [i_rs2Addr_ID](#)
rs2 address at Decode.
- rand logic< 3:0 > [i_rdAddr_EX](#)
rd address at Execute.
- rand logic< 3:0 > [i_rs1Addr_EX](#)
rs1 address at Execute.
- rand logic< 3:0 > [i_rs2Addr_EX](#)
rs2 address at Execute.
- rand logic [i_pcSrc_EX](#)
PC-redirect flag at Execute.
- rand logic< 1:0 > [i_result_src_EX](#)
Write-back source select at Execute.
- rand logic< 3:0 > [i_rdAddr_M](#)
rd address at Memory.
- rand logic [i_reg_write_M](#)
Register-file write enable at Memory.
- rand logic< 3:0 > [i_rdAddr_WB](#)
rd address at WriteBack.
- rand logic [i_reg_write_WB](#)
Register-file write enable at WriteBack.
- logic [o_stall_IF](#)
Sampled by the monitor.
- logic [o_stall_ID](#)
Sampled by the monitor.
- logic [o_flush_ID](#)
Sampled by the monitor.
- logic [o_flush_EX](#)
Sampled by the monitor.
- logic< 1:0 > [o_forward_rs1_EX](#)
Sampled by the monitor.
- logic< 1:0 > [o_forward_rs2_EX](#)
Sampled by the monitor.

8.7.1 Detailed Description

Sequence item that models one hazard-unit evaluation (stimulus + expected response fields).

Carries the fields randomized by the sequence/driver, and the result fields filled in by the monitor for the scoreboard to check against the golden reference model.

8.7.2 Member Function Documentation

8.7.2.1 new()

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_↔
DEFAULT UVM_DEFAULT uvm_object_utils_end pkt::new (
    string name = "pkt") [inline]
```

8.7.2.2 uvm_field_int() [1/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    i_pcSrc_EX ,
    UVM_DEFAULT )
```

8.7.2.3 uvm_field_int() [2/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    i_rdAddr_M ,
    UVM_DEFAULT )
```

8.7.2.4 uvm_field_int() [3/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    i_rdAddr_WB ,
    UVM_DEFAULT )
```

8.7.2.5 uvm_field_int() [4/8]

```
UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    i_rs1Addr_EX ,
    UVM_DEFAULT )
```

8.7.2.6 uvm_field_int() [5/8]

```
UVM_DEFAULT pkt::uvm_field_int (
    i_rs2Addr_ID ,
    UVM_DEFAULT )
```

8.7.2.7 uvm_field_int() [6/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT pkt::uvm↵
_field_int (
    o_flush_ID ,
    UVM_DEFAULT )
```

8.7.2.8 uvm_field_int() [7/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM↵
DEFAULT pkt::uvm_field_int (
    o_forward_rs1_EX ,
    UVM_DEFAULT )
```

8.7.2.9 uvm_field_int() [8/8]

```
UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT UVM_DEFAULT pkt::uvm_field_int (
    o_stall_IF ,
    UVM_DEFAULT )
```

8.7.2.10 uvm_object_utils_begin()

```
pkt::uvm_object_utils_begin (
    pkt )
```

8.7.3 Member Data Documentation

8.7.3.1 i_pcSrc_EX

```
rand logic pkt::i_pcSrc_EX
```

PC-redirect flag at Execute.

8.7.3.2 i_rdAddr_EX

```
rand logic<3:0> pkt::i_rdAddr_EX
```

rd address at Execute.

8.7.3.3 i_rdAddr_M

```
rand logic<3:0> pkt::i_rdAddr_M
```

rd address at Memory.

8.7.3.4 i_rdAddr_WB

```
rand logic<3:0> pkt::i_rdAddr_WB
```

rd address at WriteBack.

8.7.3.5 i_reg_write_M

```
rand logic pkt::i_reg_write_M
```

Register-file write enable at Memory.

8.7.3.6 i_reg_write_WB

```
rand logic pkt::i_reg_write_WB
```

Register-file write enable at WriteBack.

8.7.3.7 i_result_src_EX

```
rand logic<1:0> pkt::i_result_src_EX
```

Write-back source select at Execute.

8.7.3.8 i_rs1Addr_EX

```
rand logic<3:0> pkt::i_rs1Addr_EX
```

rs1 address at Execute.

8.7.3.9 i_rs1Addr_ID

```
rand logic<3:0> pkt::i_rs1Addr_ID
```

rs1 address at Decode.

8.7.3.10 i_rs2Addr_EX

```
rand logic<3:0> pkt::i_rs2Addr_EX
```

rs2 address at Execute.

8.7.3.11 i_rs2Addr_ID

```
rand logic<3:0> pkt::i_rs2Addr_ID
```

rs2 address at Decode.

8.7.3.12 o_flush_EX

```
logic pkt::o_flush_EX
```

Sampled by the monitor.

8.7.3.13 o_flush_ID

```
logic pkt::o_flush_ID
```

Sampled by the monitor.

8.7.3.14 o_forward_rs1_EX

```
logic<1:0> pkt::o_forward_rs1_EX
```

Sampled by the monitor.

8.7.3.15 o_forward_rs2_EX

```
logic<1:0> pkt::o_forward_rs2_EX
```

Sampled by the monitor.

8.7.3.16 o_stall_ID

```
logic pkt::o_stall_ID
```

Sampled by the monitor.

8.7.3.17 o_stall_IF

```
logic pkt::o_stall_IF
```

Sampled by the monitor.

The documentation for this class was generated from the following file:

- [uvm/pkt.sv](#)

8.8 s_interface Interface Reference

Hazard-unit stimulus/response interface shared by the DUT and the UVM environment.

8.8.1 Detailed Description

Hazard-unit stimulus/response interface shared by the DUT and the UVM environment.

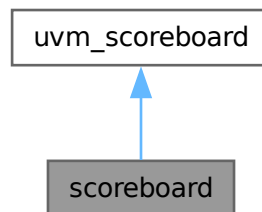
The documentation for this interface was generated from the following file:

- [tb/s_interface.sv](#)

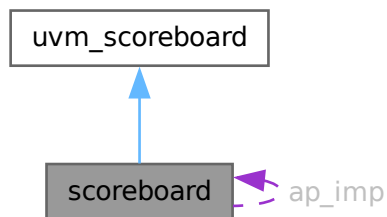
8.9 scoreboard Class Reference

UVM scoreboard for hazard-unit functional checking.

Inheritance diagram for scoreboard:



Collaboration diagram for scoreboard:



Public Member Functions

- `uvm_component_utils (scoreboard) uvm_analysis_imp< pkt`
- `new` (string name="scoreboard", uvm_component parent=null)
- virtual void `build_phase` (uvm_phase phase)
Creates the analysis imp that receives transactions from the monitor.
- virtual void `write` (pkt data)
Analysis-port callback: checks one monitored transaction against hazard_unit_golden.
- virtual void `check_phase` (uvm_phase phase)
Fails the test if any mismatch was recorded by write().
- virtual task `run_phase` (uvm_phase phase)
Run phase; unused, checking happens in write() and check_phase().

Public Attributes

- [scoreboard ap_imp](#)
Receives transactions from the monitor's analysis port.
- `int num_errors = 0`
Number of mismatches found so far.

8.9.1 Detailed Description

UVM scoreboard for hazard-unit functional checking.

Compares monitored hazard-unit inputs/outputs with the DPI-C reference model and reports mismatches as test failures.

8.9.2 Member Function Documentation

8.9.2.1 build_phase()

```
virtual void scoreboard::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Creates the analysis imp that receives transactions from the monitor.

8.9.2.2 check_phase()

```
virtual void scoreboard::check_phase (  
    uvm_phase phase) [inline], [virtual]
```

Fails the test if any mismatch was recorded by [write\(\)](#).

8.9.2.3 new()

```
scoreboard::new (  
    string name = "scoreboard",  
    uvm_component parent = null) [inline]
```

8.9.2.4 run_phase()

```
virtual task scoreboard::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Run phase; unused, checking happens in [write\(\)](#) and [check_phase\(\)](#).

8.9.2.5 uvm_component_utils()

```
scoreboard::uvm_component_utils (  
    scoreboard )
```

8.9.2.6 write()

```
virtual void scoreboard::write (  
    pkt data) [inline], [virtual]
```

Analysis-port callback: checks one monitored transaction against `hazard_unit_golden`.

8.9.3 Member Data Documentation

8.9.3.1 ap_imp

```
scoreboard scoreboard::ap_imp
```

Receives transactions from the monitor's analysis port.

8.9.3.2 num_errors

```
int scoreboard::num_errors = 0
```

Number of mismatches found so far.

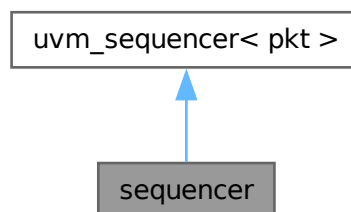
The documentation for this class was generated from the following file:

- [uvm/scoreboard.sv](#)

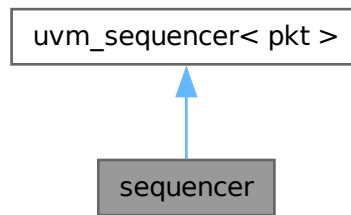
8.10 sequencer Class Reference

UVM sequencer for hazard-unit transactions.

Inheritance diagram for sequencer:



Collaboration diagram for sequencer:



Public Member Functions

- [uvm_component_utils](#) ([sequencer](#)) new(string name

Public Attributes

- `uvm_component` [parent](#)

8.10.1 Detailed Description

UVM sequencer for hazard-unit transactions.

Supplies randomized sequence items to the driver during the test run.

8.10.2 Member Function Documentation

8.10.2.1 uvm_component_utils()

```
sequencer::uvm_component_utils (
    sequencer ) [new]
```

8.10.3 Member Data Documentation

8.10.3.1 parent

```
uvm_component sequencer::parent
```

Initial value:

```
{
    super.new (name, parent)
```

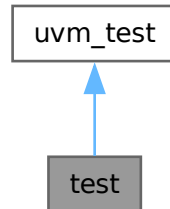
The documentation for this class was generated from the following file:

- [uvm/sequencer.sv](#)

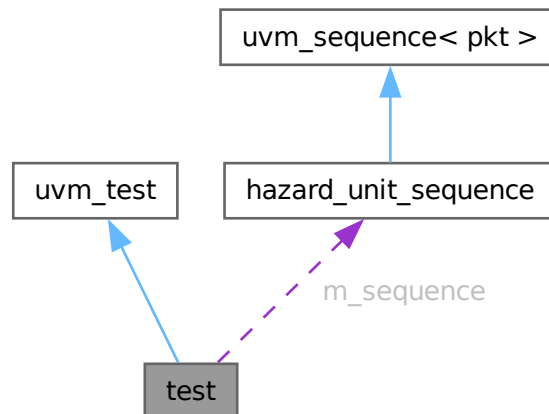
8.11 test Class Reference

Top-level UVM test for the hazard-unit verification bench.

Inheritance diagram for test:



Collaboration diagram for test:



Public Member Functions

- `uvm_component_utils (test) env m_env`
Verification environment under test.
- `new` (string name="test", uvm_component parent=null)
- virtual void `build_phase` (uvm_phase phase)
Builds the verification environment.
- virtual void `end_of_elaboration_phase` (uvm_phase phase)
Prints the component topology once elaboration is complete.
- virtual task `run_phase` (uvm_phase phase)
Starts the coverage-driven sequence and raises/drops the run-phase objection around it.

Public Attributes

- [hazard_unit_sequence m_sequence](#)
Main stimulus sequence, started on the agent's sequencer.

8.11.1 Detailed Description

Top-level UVM test for the hazard-unit verification bench.

Builds the environment, starts the main sequence, and drives the full verification flow from the test root.

8.11.2 Member Function Documentation

8.11.2.1 build_phase()

```
virtual void test::build_phase (  
    uvm_phase phase) [inline], [virtual]
```

Builds the verification environment.

8.11.2.2 end_of_elaboration_phase()

```
virtual void test::end_of_elaboration_phase (  
    uvm_phase phase) [inline], [virtual]
```

Prints the component topology once elaboration is complete.

8.11.2.3 new()

```
test::new (  
    string name = "test",  
    uvm_component parent = null) [inline]
```

8.11.2.4 run_phase()

```
virtual task test::run_phase (  
    uvm_phase phase) [inline], [virtual]
```

Starts the coverage-driven sequence and raises/drops the run-phase objection around it.

8.11.2.5 uvm_component_utils()

```
test::uvm_component_utils (  
    test )
```

Verification environment under test.

8.11.3 Member Data Documentation

8.11.3.1 m_sequence

[hazard_unit_sequence](#) test::m_sequence

Main stimulus sequence, started on the agent's sequencer.

The documentation for this class was generated from the following file:

- [uvm/test.sv](#)

Chapter 9

File Documentation

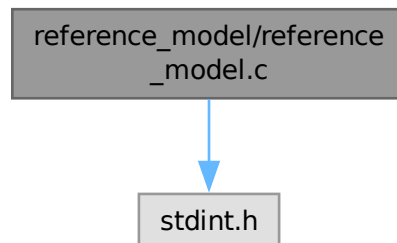
9.1 filter/README.md File Reference

9.2 reference_model/reference_model.c File Reference

Golden reference model for the hazard unit, called from the UVM scoreboard via DPI-C.

```
#include <stdint.h>
```

Include dependency graph for reference_model.c:



Functions

- void [hazard_unit_golden](#) (unsigned int rs1_id, unsigned int rs2_id, unsigned int rd_ex, unsigned int rs1_ex, unsigned int rs2_ex, unsigned char pc_src_ex, unsigned int result_src_ex, unsigned int rd_m, unsigned char reg_write_m, unsigned int rd_wb, unsigned char reg_write_wb, unsigned char *exp_stall_if, unsigned char *exp_stall_id, unsigned char *exp_flush_id, unsigned char *exp_flush_ex, unsigned int *exp_forward_rs1_ex, unsigned int *exp_forward_rs2_ex)

Computes the expected forwarding selects and stall/flush flags for one cycle.

9.2.1 Detailed Description

Golden reference model for the hazard unit, called from the UVM scoreboard via DPI-C.

Implements the forwarding and stall/flush rules expected from a standard 5-stage pipeline hazard unit (data hazards solved by forwarding, load-use hazards solved by a 1-cycle stall/bubble, control hazards solved by a flush on a taken branch/jump).

9.2.2 Function Documentation

9.2.2.1 hazard_unit_golden()

```
void hazard_unit_golden (
    unsigned int rs1_id,
    unsigned int rs2_id,
    unsigned int rd_ex,
    unsigned int rs1_ex,
    unsigned int rs2_ex,
    unsigned char pc_src_ex,
    unsigned int result_src_ex,
    unsigned int rd_m,
    unsigned char reg_write_m,
    unsigned int rd_wb,
    unsigned char reg_write_wb,
    unsigned char * exp_stall_if,
    unsigned char * exp_stall_id,
    unsigned char * exp_flush_id,
    unsigned char * exp_flush_ex,
    unsigned int * exp_forward_rs1_ex,
    unsigned int * exp_forward_rs2_ex)
```

Computes the expected forwarding selects and stall/flush flags for one cycle.

Parameters

<i>rs1_id</i>	rs1 address at the Decode stage.
<i>rs2_id</i>	rs2 address at the Decode stage.
<i>rd_ex</i>	rd address at the Execute stage.
<i>rs1_ex</i>	rs1 address at the Execute stage.
<i>rs2_ex</i>	rs2 address at the Execute stage.
<i>pc_src_ex</i>	1 if the instruction at Execute redirects the PC (branch taken/jump).
<i>result_src_ex</i>	Write-back source select of the instruction at Execute (per op_decoder.v: 00=ALU, 01=PC+4, 10=data memory, 11=PC+imm).
<i>rd_m</i>	rd address at the Memory stage.
<i>reg_write_m</i>	1 if the instruction at Memory will write the register file.
<i>rd_wb</i>	rd address at the WriteBack stage.

<i>reg_write_wb</i>	1 if the instruction at WriteBack will write the register file.
<i>exp_stall_if</i>	Output: expected o_stall_IF.
<i>exp_stall_id</i>	Output: expected o_stall_ID.
<i>exp_flush_id</i>	Output: expected o_flush_ID.
<i>exp_flush_ex</i>	Output: expected o_flush_EX.
<i>exp_forward_rs1_ex</i>	Output: expected o_forward_rs1_EX (00=none, 01=from WB, 10=from MEM).
<i>exp_forward_rs2_ex</i>	Output: expected o_forward_rs2_EX (00=none, 01=from WB, 10=from MEM).

9.3 rtl/hazard_unit.v File Reference

Functions

- always *begin if (((i_rs1Addr_EX==i_rdAddr_M) &i_reg_write_M) &(i_rs1Addr_EX !=4 'b0)) begin o_forward_rs1_EX
- end else if (((i_rs1Addr_EX==i_rdAddr_WB) &i_reg_write_WB) &(i_rs1Addr_EX !=4 'b0)) begin o_forward_rs1_EX
- end end always *begin if (((i_rs2Addr_EX==i_rdAddr_M) &i_reg_write_M) &(i_rs2Addr_EX !=4 'b0)) begin o_forward_rs2_EX
- end else if (((i_rs2Addr_EX==i_rdAddr_WB) &i_reg_write_WB) &(i_rs2Addr_EX !=4 'b0)) begin o_forward_rs2_EX

Variables

- input[REG_WIDTH-1:0] i_rs1Addr_ID
- input[REG_WIDTH-1:0] i_rs2Addr_ID
- input[REG_WIDTH-1:0] i_rdAddr_EX
- input[REG_WIDTH-1:0] i_rs1Addr_EX
- input[REG_WIDTH-1:0] i_rs2Addr_EX
- input i_pcSrc_EX
- input[1:0] i_result_src_EX
- input[REG_WIDTH-1:0] i_rdAddr_M
- input i_reg_write_M
- input[REG_WIDTH-1:0] i_rdAddr_WB
- input i_reg_write_WB
- output o_stall_IF = load_hazard_detect
- output o_stall_ID = load_hazard_detect
- output o_flush_ID = i_pcSrc_EX
- output o_flush_EX = load_hazard_detect | i_pcSrc_EX
- output reg[1:0] o_forward_rs1_EX = 2'b00
- output reg[1:0] o_forward_rs2_EX = 2'b00
- wire load_hazard_detect

9.3.1 Function Documentation

9.3.1.1 if() [1/4]

```
always *begin if (
    ((i_rs1Addr_EX==i_rdAddr_M) &i_reg_write_M) &(i_rs1Addr_EX !=4 'b0) )
```

9.3.1.2 if() [2/4]

```
end else if (
    ((i_rs1Addr_EX==i_rdAddr_WB) & i_reg_write_WB) & (i_rs1Addr_EX !=4 'b0) )
```

9.3.1.3 if() [3/4]

```
end end always *begin if (
    ((i_rs2Addr_EX==i_rdAddr_M) & i_reg_write_M) & (i_rs2Addr_EX !=4 'b0) )
```

9.3.1.4 if() [4/4]

```
end else if (
    ((i_rs2Addr_EX==i_rdAddr_WB) & i_reg_write_WB) & (i_rs2Addr_EX !=4 'b0) )
```

9.3.2 Variable Documentation

9.3.2.1 i_pcSrc_EX

```
input i_pcSrc_EX
```

9.3.2.2 i_rdAddr_EX

```
input [REG_WIDTH-1:0] i_rdAddr_EX
```

9.3.2.3 i_rdAddr_M

```
input [REG_WIDTH-1:0] i_rdAddr_M
```

9.3.2.4 i_rdAddr_WB

```
input [REG_WIDTH-1:0] i_rdAddr_WB
```

9.3.2.5 i_reg_write_M

```
input i_reg_write_M
```

9.3.2.6 i_reg_write_WB

```
input i_reg_write_WB
```

9.3.2.7 i_result_src_EX

```
input [1:0] i_result_src_EX
```

9.3.2.8 i_rs1Addr_EX

```
input [REG_WIDTH-1:0] i_rs1Addr_EX
```

9.3.2.9 i_rs1Addr_ID

```
input [REG_WIDTH-1:0] i_rs1Addr_ID
```

9.3.2.10 i_rs2Addr_EX

```
input [REG_WIDTH-1:0] i_rs2Addr_EX
```

9.3.2.11 i_rs2Addr_ID

```
input [REG_WIDTH-1:0] i_rs2Addr_ID
```

9.3.2.12 load_hazard_detect

```
end end assign load_hazard_detect
```

Initial value:

```
= ((i_result_src_EX == 2'b10) &&  
  ((i_rs1Addr_ID == i_rdAddr_EX) || (i_rs2Addr_ID == i_rdAddr_EX)))
```

9.3.2.13 o_flush_EX

```
assign o_flush_EX = load_hazard_detect | i_pcSrc_EX
```

9.3.2.14 o_flush_ID

```
assign o_flush_ID = i_pcSrc_EX
```

9.3.2.15 o_forward_rs1_EX

```
end else begin o_forward_rs1_EX = 2'b00
```

9.3.2.16 o_forward_rs2_EX

```
end else begin o_forward_rs2_EX = 2'b00
```

9.3.2.17 o_stall_ID

```
assign o_stall_ID = load_hazard_detect
```

9.3.2.18 o_stall_IF

```
assign o_stall_IF = load_hazard_detect
```

9.4 rtl/hazard_unit_wrapper.sv File Reference

Thin combinational wrapper around [hazard_unit](#), used as the DUT.

9.4.1 Detailed Description

Thin combinational wrapper around [hazard_unit](#), used as the DUT.

[hazard_unit](#) has no internal state, so the wrapper simply wires the `s_interface.dut` modport straight into it, with no clock/reset logic.

9.5 tb/s_interface.sv File Reference

Hazard-unit stimulus/response interface shared by the DUT and the UVM environment.

Functions

- interface [s_interface](#) (input logic clk, input logic reset)

9.5.1 Detailed Description

Hazard-unit stimulus/response interface shared by the DUT and the UVM environment.

Declares the pipeline-register-address/control signals consumed by the hazard unit and the stall/flush/forward signals it produces, and exposes the [driver](#), [monitor](#), and `dut` modports used by the UVM driver, the UVM monitor, and the [hazard_unit_wrapper](#) DUT respectively.

9.6 tb/tb_top.sv File Reference

Top-level testbench that instantiates the DUT, interface, and UVM test.

Functions

- `s_interface vif (.clk(clk),.reset(rst))`
- `hazard_unit_wrapper U_HAZARD_UNIT_WRAPPER (.vif(vif))`
- `run_test ("test")`

Variables

- bit `rst`
- `forever`

9.6.1 Detailed Description

Top-level testbench that instantiates the DUT, interface, and UVM test.

Connects `s_interface`, the `hazard_unit_wrapper` DUT, and the UVM test entry point in a single simulation environment.

9.6.2 Function Documentation

9.6.2.1 run_test()

```
run_test (
    "test" )
```

9.6.2.2 U_HAZARD_UNIT_WRAPPER()

```
hazard_unit_wrapper U_HAZARD_UNIT_WRAPPER (
    . vifvif)
```

9.6.2.3 vif()

```
s_interface vif (
    . clkclk,
    . resetrst)
```

9.6.3 Variable Documentation

9.6.3.1 forever

```
forever
```

Initial value:

```
{
    rst = 0
```

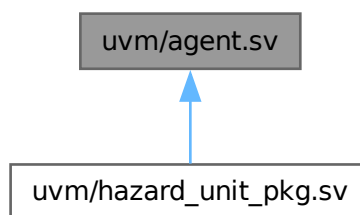
9.6.3.2 rst

```
bit rst
```

9.7 uvm/agent.sv File Reference

UVM agent that bundles the sequencer, driver, and monitor.

This graph shows which files directly or indirectly include this file:



Classes

- class [agent](#)

UVM agent that connects the sequencer, driver, and monitor.

9.7.1 Detailed Description

UVM agent that bundles the sequencer, driver, and monitor.

Implements the standard UVM agent structure for the hazard-unit verification environment, creating active and passive components as needed.

9.8 agent.sv

[Go to the documentation of this file.](#)

```

00001
00018 class agent extends uvm_agent;
00019     `uvm_component_utils (agent)
00020
00021     driver m_driver;
00022
00023     monitor m_monitor;
00024
00025     sequencer m_sequencer;
00026
00027     function new (string name = "agent", uvm_component parent = null);
00028         super.new (name, parent);
00029     endfunction
  
```

```

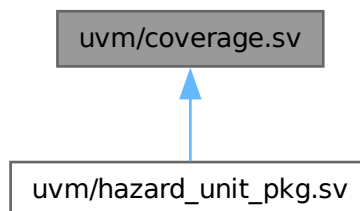
00030
00032     virtual function void build_phase (uvm_phase phase);
00033         super.build_phase (phase);
00034
00035         if (get_is_active()) begin
00036             m_sequencer = sequencer::type_id::create ("sequencer", this);
00037             m_driver = driver::type_id::create ("driver", this);
00038         end
00039
00040         m_monitor = monitor::type_id::create ("monitor", this);
00041
00042     endfunction
00043
00045     virtual function void connect_phase (uvm_phase phase);
00046         super.connect_phase (phase);
00047         if (get_is_active()) begin
00048             m_driver.seq_item_port.connect (m_sequencer.seq_item_export);
00049         end
00050     endfunction
00051
00052 endclass

```

9.9 uvm/coverage.sv File Reference

Coverage collector for hazard-unit transactions.

This graph shows which files directly or indirectly include this file:



Classes

- class `coverage`
UVM coverage subscriber for hazard-unit transaction sampling.

9.9.1 Detailed Description

Coverage collector for hazard-unit transactions.

Samples observed hazard-unit transactions and updates the coverage model used to determine whether the verification goal has been reached.

9.10 coverage.sv

[Go to the documentation of this file.](#)

```

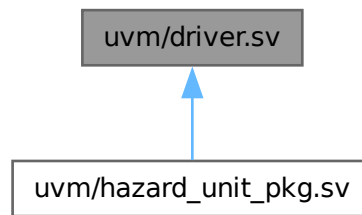
00001
00018 class coverage extends uvm_subscriber #(pkt);
00019     `uvm_component_utils(coverage)
00020
00021     pkt tr;
00022
00024     covergroup cg_hazard_unit;
00025         option.per_instance = 1;
00026
00029         cp_result_src_EX: coverpoint tr.i_result_src_EX {
00030             bins values[4] = {[0:3]};
00031         }
00032
00034         cp_pc_src_EX: coverpoint tr.i_pcSrc_EX {
00035             bins values[2] = {[0:1]};
00036         }
00037
00040         cp_load_hazard_rs1: coverpoint (tr.i_result_src_EX == 2'b10 && tr.i_rs1Addr_ID == tr.i_rdAddr_EX)
00041         {
00042             bins values[2] = {[0:1]};
00043         }
00045         cp_load_hazard_rs2: coverpoint (tr.i_result_src_EX == 2'b10 && tr.i_rs2Addr_ID == tr.i_rdAddr_EX)
00046         {
00047             bins values[2] = {[0:1]};
00048         }
00051         cp_forward_rs1_mem: coverpoint (tr.i_rs1Addr_EX == tr.i_rdAddr_M && tr.i_reg_write_M &&
00052 tr.i_rs1Addr_EX != 0) {
00053             bins values[2] = {[0:1]};
00054         }
00056         cp_forward_rs1_wb: coverpoint (tr.i_rs1Addr_EX == tr.i_rdAddr_WB && tr.i_reg_write_WB &&
00057 tr.i_rs1Addr_EX != 0) {
00058             bins values[2] = {[0:1]};
00059         }
00061         cp_forward_rs2_mem: coverpoint (tr.i_rs2Addr_EX == tr.i_rdAddr_M && tr.i_reg_write_M &&
00062 tr.i_rs2Addr_EX != 0) {
00063             bins values[2] = {[0:1]};
00064         }
00066         cp_forward_rs2_wb: coverpoint (tr.i_rs2Addr_EX == tr.i_rdAddr_WB && tr.i_reg_write_WB &&
00067 tr.i_rs2Addr_EX != 0) {
00068             bins values[2] = {[0:1]};
00069         }
00071         cp_cross_forward_rs1: cross cp_forward_rs1_mem, cp_forward_rs1_wb;
00072
00074         cp_cross_forward_rs2: cross cp_forward_rs2_mem, cp_forward_rs2_wb;
00075
00076     endgroup
00077
00078     function new(string name = "coverage", uvm_component parent);
00079         super.new(name, parent);
00080         cg_hazard_unit = new();
00081         cg_hazard_unit.set_inst_name("hazard_unit_cov");
00082     endfunction
00083
00085     virtual function void write(pkt t);
00086         this.tr = t;
00087         cg_hazard_unit.sample();
00088         uvm_config_db#(real)::set(null, "*", "cov_status", cg_hazard_unit.get_inst_coverage());
00089     endfunction
00090
00091 endclass

```

9.11 uvm/driver.sv File Reference

UVM driver that translates hazard-unit transactions into interface signal activity.

This graph shows which files directly or indirectly include this file:



Classes

- class `driver`

UVM driver for hazard-unit transactions.

9.11.1 Detailed Description

UVM driver that translates hazard-unit transactions into interface signal activity.

Receives sequence items from the sequencer and drives the pipeline-register addresses/control bits onto `s_interface` for one clock cycle each.

9.12 driver.sv

[Go to the documentation of this file.](#)

```

00001
00017 class driver extends uvm_driver #(pkt);
00018     `uvm_component_utils (driver)
00019
00020     virtual s_interface.driver vif;
00021
00022     pkt req_pkt;
00023
00024     function new (string name = "driver", uvm_component parent);
00025         super.new(name, parent);
00026     endfunction
00027
00029     virtual function void build_phase (uvm_phase phase);
00030         super.build_phase(phase);
00031         if (!uvm_config_db #(virtual s_interface)::get(this, "", "vif", vif)) begin
00032             `uvm_fatal (get_type_name (), "Didn't get handle to virtual interface if_name")
00033         end
00034     endfunction
00035
00037     virtual task run_phase(uvm_phase phase);
00038         super.run_phase(phase);
00039
00040         wait (vif.reset == 1'b1);
00041
00042         forever begin
00043             seq_item_port.get_next_item(req_pkt);
00044             drive_item(req_pkt);
00045             seq_item_port.item_done();
00046         end
00047     endtask
  
```

```

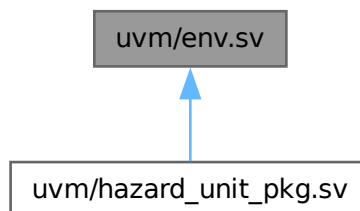
00048
00050     virtual task drive_item (pkt pkt_item);
00051         @(posedge vif.clk);
00052         vif.rs1_addr_id    <= pkt_item.i_rs1Addr_ID;
00053         vif.rs2_addr_id    <= pkt_item.i_rs2Addr_ID;
00054         vif.rd_addr_ex     <= pkt_item.i_rdAddr_EX;
00055         vif.rs1_addr_ex    <= pkt_item.i_rs1Addr_EX;
00056         vif.rs2_addr_ex    <= pkt_item.i_rs2Addr_EX;
00057         vif.pc_src_ex      <= pkt_item.i_pcSrc_EX;
00058         vif.result_src_ex  <= pkt_item.i_result_src_EX;
00059         vif.rd_addr_m      <= pkt_item.i_rdAddr_M;
00060         vif.reg_write_m    <= pkt_item.i_reg_write_M;
00061         vif.rd_addr_wb     <= pkt_item.i_rdAddr_WB;
00062         vif.reg_write_wb   <= pkt_item.i_reg_write_WB;
00063         vif.enable        <= '1;
00064
00065         @(posedge vif.clk);
00066         vif.enable <= '0;
00067     endtask
00068
00069 endclass

```

9.13 uvm/env.sv File Reference

UVM environment that instantiates the agent, scoreboard, and coverage collector.

This graph shows which files directly or indirectly include this file:



Classes

- class [env](#)

Top-level UVM environment for the hazard-unit verification bench.

9.13.1 Detailed Description

UVM environment that instantiates the agent, scoreboard, and coverage collector.

Connects the main verification components so that transactions generated by the agent can be checked and tracked by the scoreboard and coverage.

9.14 env.sv

[Go to the documentation of this file.](#)

```

00001
00016 class env extends uvm_env;
00017     `uvm_component_utils (env)
00018
00019     agent m_agent;
00020
00021     scoreboard m_scoreboard;
00022
00023     coverage m_coverage;
00024
00025     function new (string name = "env", uvm_component parent = null);
00026         super.new (name, parent);
00027     endfunction
00028
00030     virtual function void build_phase (uvm_phase phase);
00031         super.build_phase (phase);
00032         m_agent = agent::type_id::create ("agent", this);
00033         m_scoreboard = scoreboard::type_id::create ("scoreboard", this);
00034         m_coverage = coverage::type_id::create ("coverage", this);
00035     endfunction
00036
00038     virtual function void connect_phase (uvm_phase phase);
00039         super.connect_phase (phase);
00040         m_agent.m_monitor.mon_analysis_port.connect (m_scoreboard.ap_imp);
00041         m_agent.m_monitor.mon_analysis_port.connect (m_coverage.analysis_export);
00042     endfunction
00043
00044 endclass

```

9.15 uvm/hazard_unit_pkg.sv File Reference

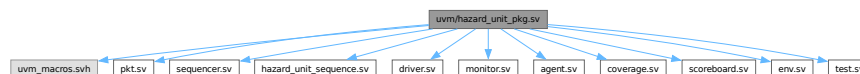
Package that includes the UVM classes used by the hazard-unit verification environment.

```

#include "uvm_macros.svh"
#include "pkt.sv"
#include "sequencer.sv"
#include "hazard_unit_sequence.sv"
#include "driver.sv"
#include "monitor.sv"
#include "agent.sv"
#include "coverage.sv"
#include "scoreboard.sv"
#include "env.sv"
#include "test.sv"

```

Include dependency graph for hazard_unit_pkg.sv:



9.15.1 Detailed Description

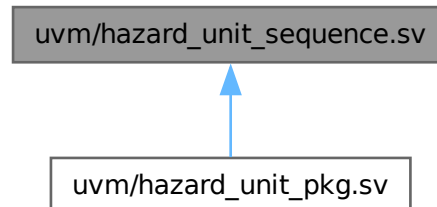
Package that includes the UVM classes used by the hazard-unit verification environment.

Centralizes the inclusion of all verification components required to build the hazard-unit UVM testbench.

9.16 uvm/hazard_unit_sequence.sv File Reference

UVM sequence that drives hazard-unit transactions until coverage reaches the target.

This graph shows which files directly or indirectly include this file:



Classes

- class [hazard_unit_sequence](#)

UVM sequence that drives hazard-unit transactions until coverage reaches the target.

9.16.1 Detailed Description

UVM sequence that drives hazard-unit transactions until coverage reaches the target.

Repeatedly sends randomized transactions while waiting for the coverage collector to report completion of the sampling process.

9.17 hazard_unit_sequence.sv

[Go to the documentation of this file.](#)

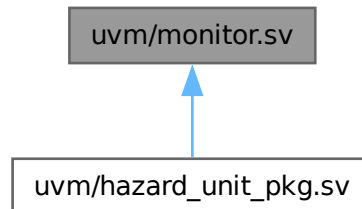
```

00001
00017 class hazard_unit_sequence extends uvm_sequence #(pkt);
00018     `uvm_object_utils(hazard_unit_sequence)
00019
00020     real current_coverage = 0;
00021
00022     int num_packets = 0;
00023
00024     function new (string name = "hazard_unit_sequence");
00025         super.new(name);
00026     endfunction
00027
00029     virtual task body();
00030         pkt packet;
00031         `uvm_create(packet)
00032         while (current_coverage < 100.0) begin
00033             `uvm_rand_send(packet)
00034             num_packets++;
00035             void' (uvm_config_db#(real)::get(null, "*", "cov_status", current_coverage));
00036             `uvm_info("SEQ", $sformatf("Status: %0.2f%%", current_coverage), UVM_LOW)
00037         end
00038         `uvm_info("SEQ", $sformatf("Total packets sent: %0d", num_packets), UVM_LOW)
00039     endtask
00040
00041 endclass
  
```

9.18 uvm/monitor.sv File Reference

UVM monitor that samples hazard-unit transactions and publishes them to subscribers.

This graph shows which files directly or indirectly include this file:



Classes

- class [monitor](#)
UVM monitor for hazard-unit transaction observation.

9.18.1 Detailed Description

UVM monitor that samples hazard-unit transactions and publishes them to subscribers.

Observes [s_interface](#) for each transaction and forwards the completed input/output fields to the scoreboard and coverage collector.

9.19 monitor.sv

[Go to the documentation of this file.](#)

```

00001
00018 class monitor extends uvm_monitor;
00019     `uvm_component_utils(monitor)
00020
00021     virtual s_interface.monitor vif;
00022
00023     uvm_analysis_port #(pkt) mon_analysis_port;
00024
00025     function new(string name = "monitor", uvm_component parent = null);
00026         super.new(name, parent);
00027     endfunction
00028
00030     virtual function void build_phase(uvm_phase phase);
00031         super.build_phase(phase);
00032         mon_analysis_port = new("mon_analysis_port", this);
00033
00034         if (!uvm_config_db #(virtual s_interface)::get(this, "", "vif", vif)) begin
00035             `uvm_error(get_type_name(), "DUT interface not found")
00036         end
00037     endfunction
00038
00049     virtual task run_phase(uvm_phase phase);
00050         super.run_phase(phase);
  
```

```

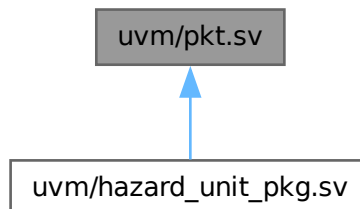
00051
00052     forever begin
00053         pkt mon_pkt;
00054         mon_pkt = pkt::type_id::create("mon_pkt", this);
00055
00056         @(posedge vif.enable);
00057         #1;
00058
00059         mon_pkt.i_rs1Addr_ID = vif.rs1_addr_id;
00060         mon_pkt.i_rs2Addr_ID = vif.rs2_addr_id;
00061         mon_pkt.i_rdAddr_EX = vif.rd_addr_ex;
00062         mon_pkt.i_rs1Addr_EX = vif.rs1_addr_ex;
00063         mon_pkt.i_rs2Addr_EX = vif.rs2_addr_ex;
00064         mon_pkt.i_pcSrc_EX = vif.pc_src_ex;
00065         mon_pkt.i_result_src_EX = vif.result_src_ex;
00066         mon_pkt.i_rdAddr_M = vif.rd_addr_m;
00067         mon_pkt.i_reg_write_M = vif.reg_write_m;
00068         mon_pkt.i_rdAddr_WB = vif.rd_addr_wb;
00069         mon_pkt.i_reg_write_WB = vif.reg_write_wb;
00070
00071         mon_pkt.o_stall_IF = vif.stall_if;
00072         mon_pkt.o_stall_ID = vif.stall_id;
00073         mon_pkt.o_flush_ID = vif.flush_id;
00074         mon_pkt.o_flush_EX = vif.flush_ex;
00075         mon_pkt.o_forward_rs1_EX = vif.forward_rs1_ex;
00076         mon_pkt.o_forward_rs2_EX = vif.forward_rs2_ex;
00077
00078         `uvm_info(get_type_name(),
00079             $sformatf("Monitored rs1_id=%0d rs2_id=%0d rd_ex=%0d rs1_ex=%0d rs2_ex=%0d
pcSrc=%0b resSrc=%0d rd_m=%0d rw_m=%0b rd_wb=%0d rw_wb=%0b -> stallIF=%0b stallID=%0b flushID=%0b
flushEX=%0b fwd1=%0d fwd2=%0d",
mon_pkt.i_rs1Addr_ID, mon_pkt.i_rs2Addr_ID, mon_pkt.i_rdAddr_EX,
mon_pkt.i_rs1Addr_EX, mon_pkt.i_rs2Addr_EX, mon_pkt.i_pcSrc_EX,
mon_pkt.i_result_src_EX, mon_pkt.i_rdAddr_M, mon_pkt.i_reg_write_M,
mon_pkt.i_rdAddr_WB, mon_pkt.i_reg_write_WB,
mon_pkt.o_stall_IF, mon_pkt.o_stall_ID, mon_pkt.o_flush_ID,
mon_pkt.o_flush_EX, mon_pkt.o_forward_rs1_EX, mon_pkt.o_forward_rs2_EX),
UVM_LOW)
00087
00088         mon_analysis_port.write(mon_pkt);
00089     end
00090 endtask
00091
00092 endclass

```

9.20 uvm/pkt.sv File Reference

Transaction item used to model one hazard-unit evaluation in the UVM environment.

This graph shows which files directly or indirectly include this file:



Classes

- class `pkt`

Sequence item that models one hazard-unit evaluation (stimulus + expected response fields).

9.20.1 Detailed Description

Transaction item used to model one hazard-unit evaluation in the UVM environment.

Encapsulates the pipeline-register addresses and control bits driven into the hazard unit, plus the stall/flush/forward fields sampled back from it.

9.21 pkt.sv

[Go to the documentation of this file.](#)

```

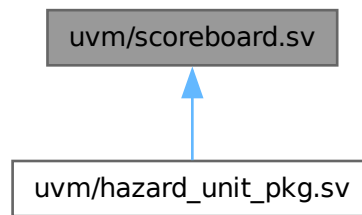
00001
00017 class pkt extends uvm_sequence_item;
00018
00019     rand logic [3:0] i_rs1Addr_ID;
00020     rand logic [3:0] i_rs2Addr_ID;
00021     rand logic [3:0] i_rdAddr_EX;
00022     rand logic [3:0] i_rs1Addr_EX;
00023     rand logic [3:0] i_rs2Addr_EX;
00024     rand logic      i_pcSrc_EX;
00025     rand logic [1:0] i_result_src_EX;
00026     rand logic [3:0] i_rdAddr_M;
00027     rand logic      i_reg_write_M;
00028     rand logic [3:0] i_rdAddr_WB;
00029     rand logic      i_reg_write_WB;
00030
00031     logic      o_stall_IF;
00032     logic      o_stall_ID;
00033     logic      o_flush_ID;
00034     logic      o_flush_EX;
00035     logic [1:0] o_forward_rs1_EX;
00036     logic [1:0] o_forward_rs2_EX;
00037
00038     `uvm_object_utils_begin(pkt)
00039         `uvm_field_int (i_rs1Addr_ID, UVM_DEFAULT)
00040         `uvm_field_int (i_rs2Addr_ID, UVM_DEFAULT)
00041         `uvm_field_int (i_rdAddr_EX, UVM_DEFAULT)
00042         `uvm_field_int (i_rs1Addr_EX, UVM_DEFAULT)
00043         `uvm_field_int (i_rs2Addr_EX, UVM_DEFAULT)
00044         `uvm_field_int (i_pcSrc_EX, UVM_DEFAULT)
00045         `uvm_field_int (i_result_src_EX, UVM_DEFAULT)
00046         `uvm_field_int (i_rdAddr_M, UVM_DEFAULT)
00047         `uvm_field_int (i_reg_write_M, UVM_DEFAULT)
00048         `uvm_field_int (i_rdAddr_WB, UVM_DEFAULT)
00049         `uvm_field_int (i_reg_write_WB, UVM_DEFAULT)
00050         `uvm_field_int (o_stall_IF, UVM_DEFAULT)
00051         `uvm_field_int (o_stall_ID, UVM_DEFAULT)
00052         `uvm_field_int (o_flush_ID, UVM_DEFAULT)
00053         `uvm_field_int (o_flush_EX, UVM_DEFAULT)
00054         `uvm_field_int (o_forward_rs1_EX, UVM_DEFAULT)
00055         `uvm_field_int (o_forward_rs2_EX, UVM_DEFAULT)
00056     `uvm_object_utils_end
00057
00058     function new(string name = "pkt");
00059         super.new(name);
00060     endfunction
00061
00062 endclass

```

9.22 uvm/scoreboard.sv File Reference

Scoreboard that checks hazard-unit transactions against the golden reference model.

This graph shows which files directly or indirectly include this file:



Classes

- class [scoreboard](#)

UVM scoreboard for hazard-unit functional checking.

Functions

- void [hazard_unit_golden](#) (input int unsigned rs1_id, input int unsigned rs2_id, input int unsigned rd_ex, input int unsigned rs1_ex, input int unsigned rs2_ex, input byte unsigned pc_src_ex, input int unsigned result_src_ex, input int unsigned rd_m, input byte unsigned reg_write_m, input int unsigned rd_wb, input byte unsigned reg_write_wb, output byte unsigned exp_stall_if, output byte unsigned exp_stall_id, output byte unsigned exp_flush_id, output byte unsigned exp_flush_ex, output int unsigned exp_forward_rs1_ex, output int unsigned exp_forward_rs2_ex)

DPI-C import of the golden reference model (see [reference_model.c](#)).

9.22.1 Detailed Description

Scoreboard that checks hazard-unit transactions against the golden reference model.

Receives monitored transactions and validates them against the DPI-C reference model implemented in the C source file.

9.22.2 Function Documentation

9.22.2.1 hazard_unit_golden()

```

void hazard_unit_golden (
    input int unsigned rs1_id,
    input int unsigned rs2_id,
    input int unsigned rd_ex,
    input int unsigned rs1_ex,
    input int unsigned rs2_ex,
    input byte unsigned pc_src_ex,
    input int unsigned result_src_ex,

```

```

input int unsigned rd_m,
input byte unsigned reg_write_m,
input int unsigned rd_wb,
input byte unsigned reg_write_wb,
output byte unsigned exp_stall_if,
output byte unsigned exp_stall_id,
output byte unsigned exp_flush_id,
output byte unsigned exp_flush_ex,
output int unsigned exp_forward_rs1_ex,
output int unsigned exp_forward_rs2_ex)

```

DPI-C import of the golden reference model (see [reference_model.c](#)).

Parameters

<i>rs1_id</i>	rs1 address at Decode.
<i>rs2_id</i>	rs2 address at Decode.
<i>rd_ex</i>	rd address at Execute.
<i>rs1_ex</i>	rs1 address at Execute.
<i>rs2_ex</i>	rs2 address at Execute.
<i>pc_src_ex</i>	PC-redirect flag at Execute.
<i>result_src_ex</i>	Write-back source select at Execute.
<i>rd_m</i>	rd address at Memory.
<i>reg_write_m</i>	Register-file write enable at Memory.
<i>rd_wb</i>	rd address at WriteBack.
<i>reg_write_wb</i>	Register-file write enable at WriteBack.
<i>exp_stall_if</i>	Output: expected o_stall_IF.
<i>exp_stall_id</i>	Output: expected o_stall_ID.
<i>exp_flush_id</i>	Output: expected o_flush_ID.
<i>exp_flush_ex</i>	Output: expected o_flush_EX.
<i>exp_forward_rs1_ex</i>	Output: expected o_forward_rs1_EX.
<i>exp_forward_rs2_ex</i>	Output: expected o_forward_rs2_EX.

9.23 scoreboard.sv

[Go to the documentation of this file.](#)

```

00001
00030 import "DPI-C" function void hazard_unit_golden(
00031     input  int unsigned rs1_id,
00032     input  int unsigned rs2_id,
00033     input  int unsigned rd_ex,
00034     input  int unsigned rs1_ex,
00035     input  int unsigned rs2_ex,
00036     input  byte unsigned pc_src_ex,
00037     input  int unsigned result_src_ex,
00038     input  int unsigned rd_m,

```

```

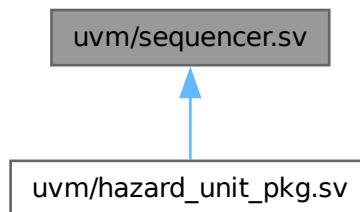
00039 input byte unsigned reg_write_m,
00040 input int unsigned rd_wb,
00041 input byte unsigned reg_write_wb,
00042 output byte unsigned exp_stall_if,
00043 output byte unsigned exp_stall_id,
00044 output byte unsigned exp_flush_id,
00045 output byte unsigned exp_flush_ex,
00046 output int unsigned exp_forward_rs1_ex,
00047 output int unsigned exp_forward_rs2_ex
00048 );
00049
00057 class scoreboard extends uvm_scoreboard;
00058     `uvm_component_utils(scoreboard)
00059
00060     uvm_analysis_imp #(pkt, scoreboard) ap_imp;
00061
00062     int num_errors = 0;
00063
00064     function new(string name = "scoreboard", uvm_component parent = null);
00065         super.new(name, parent);
00066     endfunction
00067
00069     virtual function void build_phase(uvm_phase phase);
00070         super.build_phase(phase);
00071         ap_imp = new("ap_imp", this);
00072     endfunction
00073
00075     virtual function void write(pkt data);
00076         byte unsigned exp_stall_if_raw, exp_stall_id_raw, exp_flush_id_raw, exp_flush_ex_raw;
00077         logic [1:0] exp_forward_rs1_ex, exp_forward_rs2_ex;
00078         logic exp_stall_if, exp_stall_id, exp_flush_id, exp_flush_ex;
00079
00080         hazard_unit_golden(data.i_rs1Addr_ID, data.i_rs2Addr_ID, data.i_rdAddr_EX,
00081             data.i_rs1Addr_EX, data.i_rs2Addr_EX, data.i_pcSrc_EX, data.i_result_src_EX,
00082             data.i_rdAddr_M, data.i_reg_write_M, data.i_rdAddr_WB, data.i_reg_write_WB,
00083             exp_stall_if_raw, exp_stall_id_raw, exp_flush_id_raw, exp_flush_ex_raw,
00084             exp_forward_rs1_ex, exp_forward_rs2_ex);
00085
00086         exp_stall_if = exp_stall_if_raw[0];
00087         exp_stall_id = exp_stall_id_raw[0];
00088         exp_flush_id = exp_flush_id_raw[0];
00089         exp_flush_ex = exp_flush_ex_raw[0];
00090
00091         if (data.o_stall_IF == exp_stall_if && data.o_stall_ID == exp_stall_id &&
00092             data.o_flush_ID == exp_flush_id && data.o_flush_EX == exp_flush_ex &&
00093             data.o_forward_rs1_EX == exp_forward_rs1_ex && data.o_forward_rs2_EX == exp_forward_rs2_ex)
00094             begin
00095                 `uvm_info("SCOREBOARD",
00096                     $sformatf("PASS: stallIF=%0b stallID=%0b flushID=%0b flushEX=%0b fwd1=%0d fwd2=%0d",
00097                         data.o_stall_IF, data.o_stall_ID, data.o_flush_ID, data.o_flush_EX,
00098                         data.o_forward_rs1_EX, data.o_forward_rs2_EX),
00099                     UVM_LOW)
00100             end
00101             else begin
00102                 `uvm_error("SCOREBOARD",
00103                     $sformatf("FAIL: rs1_id=%0d rs2_id=%0d rd_ex=%0d rs1_ex=%0d rs2_ex=%0d pcSrc=%0b resSrc=%0d
00104 rd_m=%0d rw_m=%0b rd_wb=%0d rw_wb=%0b EXPECTED stallIF=%0b stallID=%0b flushID=%0b flushEX=%0b
00105 fwd1=%0d fwd2=%0d GOT stallIF=%0b stallID=%0b flushID=%0b flushEX=%0b fwd1=%0d fwd2=%0d",
00106                     data.i_rs1Addr_ID, data.i_rs2Addr_ID, data.i_rdAddr_EX, data.i_rs1Addr_EX,
00107                     data.i_rs2Addr_EX,
00108                     data.i_pcSrc_EX, data.i_result_src_EX, data.i_rdAddr_M, data.i_reg_write_M,
00109                     data.i_rdAddr_WB, data.i_reg_write_WB,
00110                     exp_stall_if, exp_stall_id, exp_flush_id, exp_flush_ex, exp_forward_rs1_ex,
00111                     exp_forward_rs2_ex,
00112                     data.o_stall_IF, data.o_stall_ID, data.o_flush_ID, data.o_flush_EX,
00113                     data.o_forward_rs1_EX, data.o_forward_rs2_EX))
00114                 this.num_errors++;
00115             end
00116         endfunction
00117
00114     virtual function void check_phase(uvm_phase phase);
00115         super.check_phase(phase);
00116         if (this.num_errors > 0) begin
00117             `uvm_fatal("FINAL_RESULT",
00118                 $sformatf("TEST FAILED: Scoreboard found %0d errors.", num_errors))
00119         end
00120         else begin
00121             `uvm_info("FINAL_RESULT", "TEST PASS: All transactions were correct.", UVM_NONE)
00122         end
00123     endfunction
00124
00126     virtual task run_phase(uvm_phase phase);
00127         super.run_phase(phase);
00128     endtask
00129
00130 endclass

```

9.24 uvm/sequencer.sv File Reference

UVM sequencer that provides transactions to the driver.

This graph shows which files directly or indirectly include this file:



Classes

- class [sequencer](#)
UVM sequencer for hazard-unit transactions.

9.24.1 Detailed Description

UVM sequencer that provides transactions to the driver.

Acts as the transaction source for the UVM driver in the hazard-unit verification.

9.25 sequencer.sv

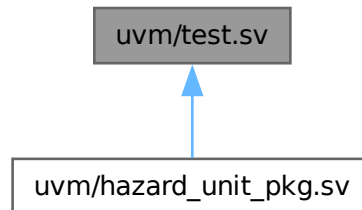
[Go to the documentation of this file.](#)

```
00001
00014 class sequencer extends uvm_sequencer #(pkt);
00015
00016     `uvm_component_utils (sequencer)
00017
00018     function new (string name="sequencer", uvm_component parent);
00019         super.new (name, parent);
00020     endfunction
00021
00022 endclass
```

9.26 uvm/test.sv File Reference

Top-level UVM test that builds the environment and starts the main sequence.

This graph shows which files directly or indirectly include this file:



Classes

- class `test`

Top-level UVM test for the hazard-unit verification bench.

9.26.1 Detailed Description

Top-level UVM test that builds the environment and starts the main sequence.

Instantiates the verification environment and launches the hazard-unit sequence that drives the DUT under test.

9.27 test.sv

[Go to the documentation of this file.](#)

```

00001
00016 class test extends uvm_test;
00017     `uvm_component_utils (test)
00018
00019     env m_env;
00020
00021     hazard_unit_sequence m_sequence;
00022
00023     function new (string name = "test", uvm_component parent = null);
00024         super.new (name, parent);
00025     endfunction
00026
00028     virtual function void build_phase (uvm_phase phase);
00029         super.build_phase (phase);
00030         m_env = env::type_id::create ("env", this);
00031     endfunction
00032
00034     virtual function void end_of_elaboration_phase (uvm_phase phase);
00035         uvm_root::get().print_topology();
00036     endfunction
00037
00039     virtual task run_phase(uvm_phase phase);
00040         super.run_phase(phase);
00041
00042         m_sequence = hazard_unit_sequence::type_id::create("sequence");
00043
00044         phase.raise_objection(this);
00045         m_sequence.start(m_env.m_agent.m_sequencer);
00046         phase.drop_objection(this);
00047     endtask
00048 endclass
  
```

Index

- agent, [17](#)
 - build_phase, [18](#)
 - connect_phase, [18](#)
 - m_monitor, [19](#)
 - m_sequencer, [19](#)
 - new, [18](#)
 - uvm_component_utils, [18](#)
- ap_imp
 - scoreboard, [38](#)
- body
 - hazard_unit_sequence, [27](#)
- build_phase
 - agent, [18](#)
 - driver, [23](#)
 - env, [25](#)
 - monitor, [29](#)
 - scoreboard, [37](#)
 - test, [41](#)
- cg_hazard_unit
 - coverage, [20](#)
- check_phase
 - scoreboard, [37](#)
- connect_phase
 - agent, [18](#)
 - env, [25](#)
- coverage, [19](#)
 - cg_hazard_unit, [20](#)
 - new, [20](#)
 - uvm_component_utils, [21](#)
 - write, [21](#)
- drive_item
 - driver, [23](#)
- driver, [21](#)
 - build_phase, [23](#)
 - drive_item, [23](#)
 - new, [23](#)
 - req_pkt, [23](#)
 - run_phase, [23](#)
 - uvm_component_utils, [23](#)
- end_of_elaboration_phase
 - test, [41](#)
- env, [24](#)
 - build_phase, [25](#)
 - connect_phase, [25](#)
 - m_coverage, [25](#)
 - m_scoreboard, [25](#)
 - new, [25](#)
 - uvm_component_utils, [25](#)
- filter Directory Reference, [15](#)
- filter/README.md, [43](#)
- forever
 - tb_top.sv, [49](#)
- hazard_unit.v
 - i_pcSrc_EX, [46](#)
 - i_rdAddr_EX, [46](#)
 - i_rdAddr_M, [46](#)
 - i_rdAddr_WB, [46](#)
 - i_reg_write_M, [46](#)
 - i_reg_write_WB, [46](#)
 - i_result_src_EX, [46](#)
 - i_rs1Addr_EX, [47](#)
 - i_rs1Addr_ID, [47](#)
 - i_rs2Addr_EX, [47](#)
 - i_rs2Addr_ID, [47](#)
 - if, [45](#), [46](#)
 - load_hazard_detect, [47](#)
 - o_flush_EX, [47](#)
 - o_flush_ID, [47](#)
 - o_forward_rs1_EX, [47](#)
 - o_forward_rs2_EX, [47](#)
 - o_stall_ID, [47](#)
 - o_stall_IF, [48](#)
- hazard_unit_golden
 - reference_model.c, [44](#)
 - scoreboard.sv, [60](#)
- hazard_unit_sequence, [26](#)
 - body, [27](#)
 - new, [27](#)
 - num_packets, [27](#)
 - uvm_object_utils, [27](#)
- i_pcSrc_EX
 - hazard_unit.v, [46](#)
 - pkt, [33](#)
- i_rdAddr_EX
 - hazard_unit.v, [46](#)
 - pkt, [33](#)
- i_rdAddr_M
 - hazard_unit.v, [46](#)
 - pkt, [33](#)
- i_rdAddr_WB
 - hazard_unit.v, [46](#)
 - pkt, [33](#)
- i_reg_write_M

- hazard_unit.v, 46
- pkt, 33
- i_reg_write_WB
 - hazard_unit.v, 46
 - pkt, 33
- i_result_src_EX
 - hazard_unit.v, 46
 - pkt, 34
- i_rs1Addr_EX
 - hazard_unit.v, 47
 - pkt, 34
- i_rs1Addr_ID
 - hazard_unit.v, 47
 - pkt, 34
- i_rs2Addr_EX
 - hazard_unit.v, 47
 - pkt, 34
- i_rs2Addr_ID
 - hazard_unit.v, 47
 - pkt, 34
- if
 - hazard_unit.v, 45, 46
- Interfaces, 12
 - s_interface, 12
- load_hazard_detect
 - hazard_unit.v, 47
- m_coverage
 - env, 25
- m_monitor
 - agent, 19
- m_scoreboard
 - env, 25
- m_sequence
 - test, 41
- m_sequencer
 - agent, 19
- Modules, 11
- mon_analysis_port
 - monitor, 29
- monitor, 28
 - build_phase, 29
 - mon_analysis_port, 29
 - new, 29
 - run_phase, 29
 - uvm_component_utils, 29
- new
 - agent, 18
 - coverage, 20
 - driver, 23
 - env, 25
 - hazard_unit_sequence, 27
 - monitor, 29
 - pkt, 31
 - scoreboard, 37
 - test, 41
- num_errors
 - scoreboard, 38
- num_packets
 - hazard_unit_sequence, 27
- o_flush_EX
 - hazard_unit.v, 47
 - pkt, 34
- o_flush_ID
 - hazard_unit.v, 47
 - pkt, 34
- o_forward_rs1_EX
 - hazard_unit.v, 47
 - pkt, 34
- o_forward_rs2_EX
 - hazard_unit.v, 47
 - pkt, 35
- o_stall_ID
 - hazard_unit.v, 47
 - pkt, 35
- o_stall_IF
 - hazard_unit.v, 48
 - pkt, 35
- parent
 - sequencer, 39
- pkt, 30
 - i_pcSrc_EX, 33
 - i_rdAddr_EX, 33
 - i_rdAddr_M, 33
 - i_rdAddr_WB, 33
 - i_reg_write_M, 33
 - i_reg_write_WB, 33
 - i_result_src_EX, 34
 - i_rs1Addr_EX, 34
 - i_rs1Addr_ID, 34
 - i_rs2Addr_EX, 34
 - i_rs2Addr_ID, 34
 - new, 31
 - o_flush_EX, 34
 - o_flush_ID, 34
 - o_forward_rs1_EX, 34
 - o_forward_rs2_EX, 35
 - o_stall_ID, 35
 - o_stall_IF, 35
 - uvm_field_int, 31, 32
 - uvm_object_utils_begin, 33
- reference_model Directory Reference, 15
- reference_model.c
 - hazard_unit_golden, 44
- reference_model/reference_model.c, 43
- req_pkt
 - driver, 23
- rst
 - tb_top.sv, 49
- RTL design, 11
- rtl Directory Reference, 15
- rtl/hazard_unit.v, 45
- rtl/hazard_unit_wrapper.sv, 48

- run_phase
 - driver, 23
 - monitor, 29
 - scoreboard, 37
 - test, 41
- run_test
 - tb_top.sv, 49
- s_interface, 35
 - Interfaces, 12
- scoreboard, 36
 - ap_imp, 38
 - build_phase, 37
 - check_phase, 37
 - new, 37
 - num_errors, 38
 - run_phase, 37
 - uvm_component_utils, 37
 - write, 37
- scoreboard.sv
 - hazard_unit_golden, 60
- sequencer, 38
 - parent, 39
 - uvm_component_utils, 39
- tb Directory Reference, 16
- tb/s_interface.sv, 48
- tb/tb_top.sv, 48
- tb_top.sv
 - forever, 49
 - rst, 49
 - run_test, 49
 - U_HAZARD_UNIT_WRAPPER, 49
 - vif, 49
- test, 40
 - build_phase, 41
 - end_of_elaboration_phase, 41
 - m_sequence, 41
 - new, 41
 - run_phase, 41
 - uvm_component_utils, 41
- Testbench components, 11
- U_HAZARD_UNIT_WRAPPER
 - tb_top.sv, 49
- uvm Directory Reference, 16
- UVM verification components, 13
- uvm/agent.sv, 50
- uvm/coverage.sv, 51, 52
- uvm/driver.sv, 52, 53
- uvm/env.sv, 54, 55
- uvm/hazard_unit_pkg.sv, 55
- uvm/hazard_unit_sequence.sv, 56
- uvm/monitor.sv, 57
- uvm/pkt.sv, 58, 59
- uvm/scoreboard.sv, 59, 61
- uvm/sequencer.sv, 63
- uvm/test.sv, 64
- uvm_component_utils
 - agent, 18
 - coverage, 21
 - driver, 23
 - env, 25
 - monitor, 29
 - scoreboard, 37
 - sequencer, 39
 - test, 41
- uvm_field_int
 - pkt, 31, 32
- uvm_object_utils
 - hazard_unit_sequence, 27
- uvm_object_utils_begin
 - pkt, 33
- vif
 - tb_top.sv, 49
- write
 - coverage, 21
 - scoreboard, 37